

Μεταγλωττιστές 2024-25

Προσθήκη συναρτήσεων στη γραμματική
των αριθμητικών εκφράσεων

Στόχος

- Η υποστήριξη απλών συναρτήσεων από τον συντακτικό αναλυτή και AST interpreter
 - Δηλώνονται στην **αρχή** του κώδικα

function func-name (argument-list) block-statement

- Το κυρίως πρόγραμμα ακολουθεί **μετά τις δηλώσεις των συναρτήσεων**
- Καλούνται ως **expressions** (ως μέρος μιας έκφρασης)

func-name (expression-list)

- Στο δεξιό μέρος του = ή μετά το print
- Επιστρέφουν μια float τιμή

return expression

- Το return επιτρέπεται ακόμα και στο κυρίως πρόγραμμα

Προσθήκες στη γραμματική (1)

```
Program → F_declarations Stmt_list  
F_declarations → F_declaration F_declarations | ε  
F_declaration → function id ( Param_list ) Block_stmt  
Param_list → id Param_tail | ε  
Param_tail → , id Param_tail | ε
```

- **1^ο μέρος:** προσθήκη πριν το Stmt_list
 - **F_declarations:** Προηγούνται οι δηλώσεις των συναρτήσεων (0 ή περισσότερες)
 - **Stmt_list:** Το κυρίως πρόγραμμα που ακολουθεί (μπορεί επίσης να είναι κενό)
 - **Param_list:** μηδέν ή περισσότερες παράμετροι στον ορισμό μιας συνάρτησης

Προσθήκες στη γραμματική (2)

```
Stmt → id = Lexpr | print Lexpr  
      | if Lexpr Block_stmt (else Block_stmt)?  
      | while Lexpr Block_stmt  
      | return Lexpr
```

- 2^ο μέρος: προσθήκες στο Stmt
 - Προσθήκη εντολής (statement) return

Προσθήκες στη γραμματική (3)

$\text{Factor} \rightarrow (\text{Lexpr}) \mid \text{id Id_or_call} \mid \text{number}$

$\text{Id_or_call} \rightarrow (\text{Arg_list}) \mid \varepsilon$

$\text{Arg_list} \rightarrow \text{Lexpr Arg_tail} \mid \varepsilon$

$\text{Arg_tail} \rightarrow , \text{Lexpr Arg_tail} \mid \varepsilon$

- 3^ο μέρος: προσθήκες στο Factor
 - Για την υποστήριξη της κλήσης συναρτήσεων ως μέρη έκφρασης (**expression**)
 - **Arg_list**: μηδέν ή περισσότερα ορίσματα (εκφράσεις Lexpr) στην κλήση μιας συνάρτησης

Μη τερματικό	Σύνολο FIRST	Σύνολο FOLLOW
Program	function id print if while return	#
F_declarations	function	id print if while return #
F_declaration	function	
Param_list	id	)
Param_tail	,	)
Stmt_list	id print if while return	} #
Stmt	id print if while return	
Block_stmt	{ id print if while return	
Lexpr	not (id number	
Lterm_tail	or	) { } , function id print if else while return #
Lterm	not (id number	

Μη τερματικό	Σύνολο FIRST	Σύνολο FOLLOW
Lfactor_tail	and	) { } , function or id print if else while return #
Lfactor	not (id number	
Rest	== != > >= < <=	) { } , function and or id print if else while return #
Expr	(id number	
Term	(id number	
Factor	(id number	
Id_or_call	(	* / + - == != > >= < <= and or { } ,) function id print if while return else #
Arg_list	not (id num	)
Arg_tail	,	)
Addop	+ -	
Multop	* /	
Relop	== != > >= < <=	

Νέα tokens: , function return

Δοκιμαστική είσοδος

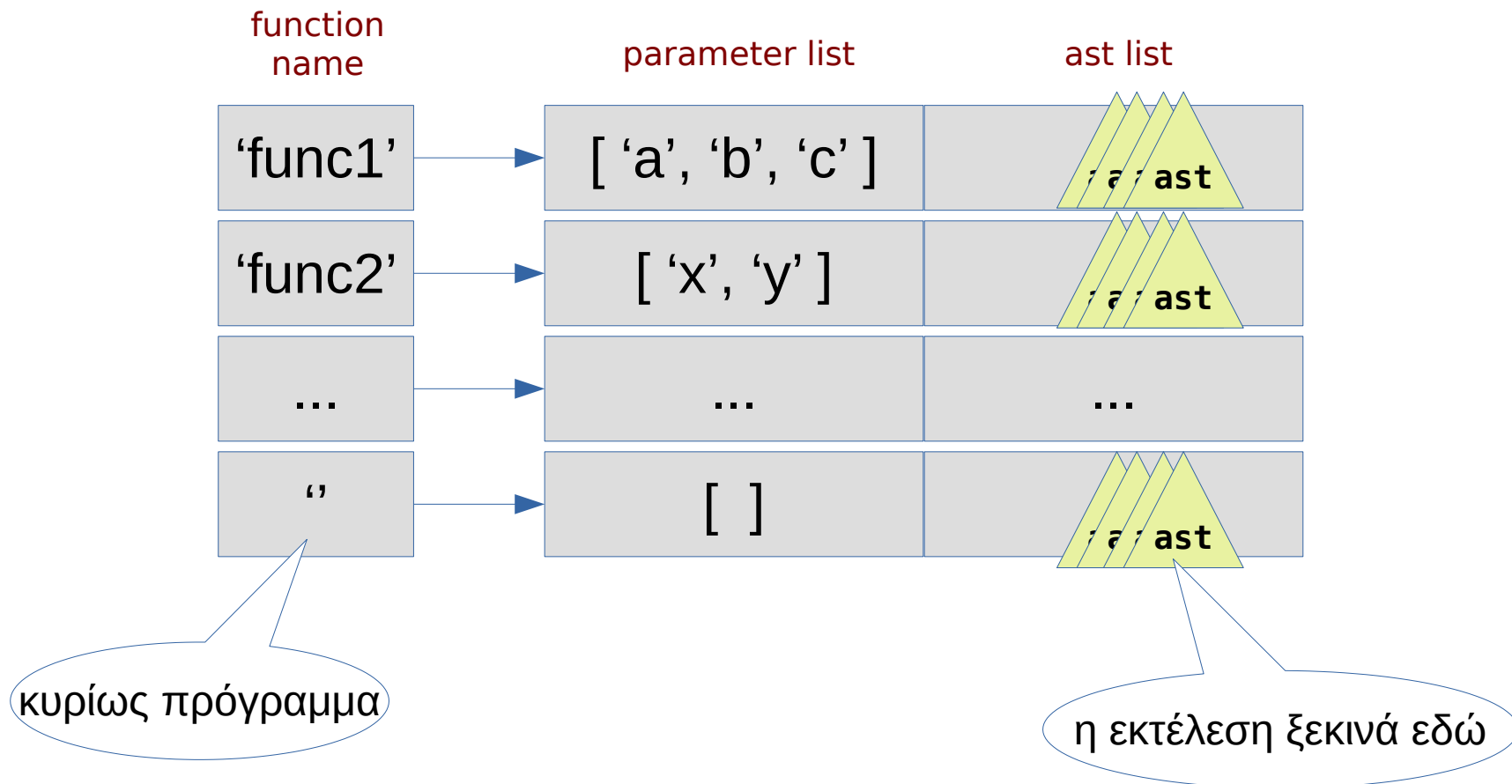
```
function um(x)
    return 0-x
function cube(x) {
    return x*x*x
}
a = 2 + 7.55*44
print a
if a-7!=3 or a<355 {
    b = 3*(a-99.01)
    it = 5
    while it>0 {
        print cube(um(it+b*0.23))
        it = it - 1
    }
}
else
    print a-3.14
```


Δηλώσεις συναρτήσεων

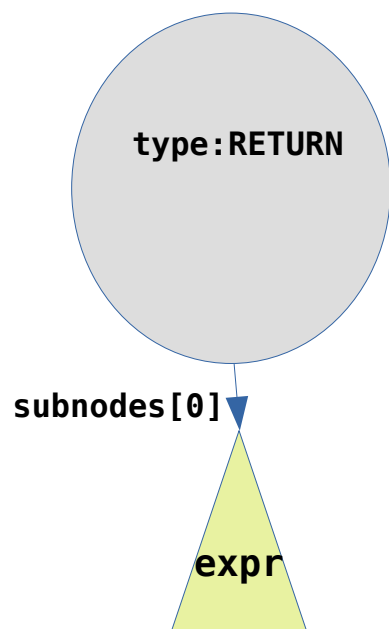
- Η συντακτική ανάλυση της δήλωσης μιας συνάρτησης δημιουργεί μια νέα **λίστα με AST εντολών**
- Η πληροφορία κάθε συνάρτησης περιέχει επίσης μια **λίστα με τα ονόματα των παραμέτρων** της
- Οι δύο αυτές πληροφορίες αποθηκεύονται σε ένα python dict (πίνακα συναρτήσεων) για **κάθε συνάρτηση** που δηλώνεται, καθώς και για το **κυρίως πρόγραμμα**
 - Ο συντακτικός αναλυτής θα πρέπει να προειδοποιεί μέσω σφάλματος για **πολλαπλές δηλώσεις** της ίδιας συνάρτησης

Πίνακας συναρτήσεων

- Ένα python dict με την πληροφορία για κάθε συνάρτηση
 - Το κυρίως πρόγραμμα αποθηκεύεται ως συνάρτηση " (κενό string)

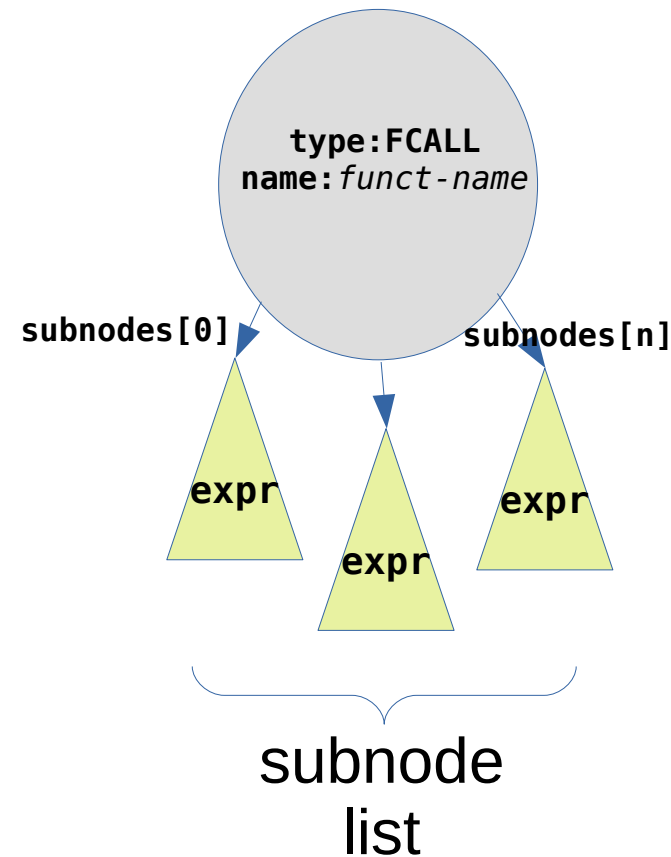


Κόμβος AST για το return



- Ο τύπος του κόμβου είναι RETURN
- Υπάρχει μοναδικό παιδί (subnodes[0]) με ένα υποδέντρο αριθμητικής έκφρασης (Lexpr)
 - Υπολογίζει την επιστρεφόμενη τιμή

Κόμβος AST για την κλήση συνάρτησης



- Ο τύπος του κόμβου είναι FCALL
- Το attribute “name” περιέχει το όνομα της καλούμενης συνάρτησης
- Τα παιδιά (subnodes) του κόμβου είναι μια σειρά από υποδέντρα αριθμητικής έκφρασης, ένα για κάθε όρισμα κλήσης
 - με τη σειρά που αντιστοιχούν στις δηλωμένες παραμέτρους της συνάρτησης

Ορθή κλήση συναρτήσεων

- Ο έλεγχος ύπαρξης της καλούμενης συνάρτησης και του σωστού αριθμού ορισμάτων μετατίθεται στη φάση της εκτέλεσης (στον interpreter)
 - Δεν υποστηρίζονται “forward declarations”
 - Κατά τη συντακτική ανάλυση μιας κλήσης συνάρτησης είναι πιθανό η καλούμενη συνάρτηση να μην έχει δηλωθεί ακόμα
- Με τον ίδιο τρόπο, η ύπαρξη ή όχι επιστρεφόμενης τιμής μετατίθεται επίσης στη φάση εκτέλεσης
 - Αν δηλαδή χρησιμοποιήθηκε εντολή **return** πριν το τέλος της συνάρτησης

AST Interpreter

- Προσθήκη λειτουργικότητας για την **κλήση συναρτήσεων** (κόμβοι FCALL)
 - Απαιτείται επιστρεφόμενη τιμή
- Προσθήκη λειτουργικότητας για την **επιστροφή τιμών** από συναρτήσεις (κόμβοι RETURN)
 - Και έλεγχος της **ροής εκτέλεσης**
 - Μετά από return **δεν πρέπει να εκτελεστούν οι επόμενες εντολές**

Εμβέλεια μεταβλητών

- Στη γλώσσα μας για λόγους απλότητας θα υπάρχουν **μόνο τοπικές μεταβλητές**
 - Κάθε συνάρτηση θα έχει τις δικές της μεταβλητές
 - Για την ακρίβεια: κάθε κλήση μιας συνάρτησης θα έχει τις δικές της μεταβλητές
 - Η μόνη επικοινωνία μεταξύ συναρτήσεων και του κυρίως προγράμματος θα γίνεται **μέσω παραμέτρων και επιστρεφόμενων τιμών**
 - **Δεν** θα υποστηρίζονται global μεταβλητές

Χώρος αποθήκευσης τοπικών μεταβλητών

- Όλες οι «δομημένες» γλώσσες προγραμματισμού χρησιμοποιούν μια στοίβα στη μνήμη (**runtime stack**) για την αποθήκευση των τοπικών μεταβλητών τους
- Στη στοίβα δεσμεύεται χώρος για τις **τοπικές μεταβλητές** σε κάθε κλήση συνάρτησης (**call frame** ή activation record)
- Στη επιστροφή από μια κλήση συνάρτησης το τρέχον call frame **αποδεσμεύεται**
 - Η στοίβα επιστρέφει στην κατάσταση **πριν** την τελευταία κλήση
- Στα call frames αποθηκεύονται επίσης οι **παράμετροι εισόδου** στη συνάρτηση και η **επιστρεφόμενη τιμή**
 - Σημείωση: οι σύγχρονοι μεταγλωττιστές προσπαθούν μέσω βελτιστοποιήσεων να κρατούν μεγάλο μέρος από τα παραπάνω σε **καταχωρητές** αντί της στοίβας (κύρια μνήμη)

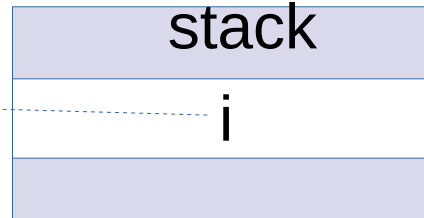
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

runtime



τρέχον call
frame
κλήσης f()

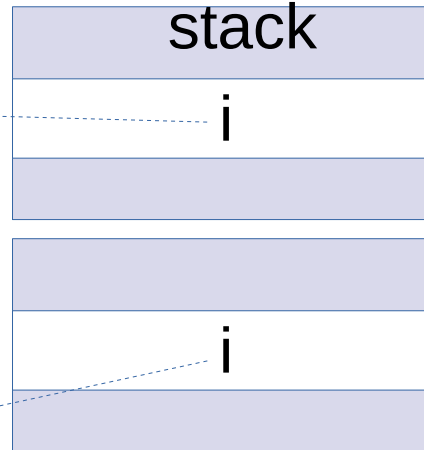
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

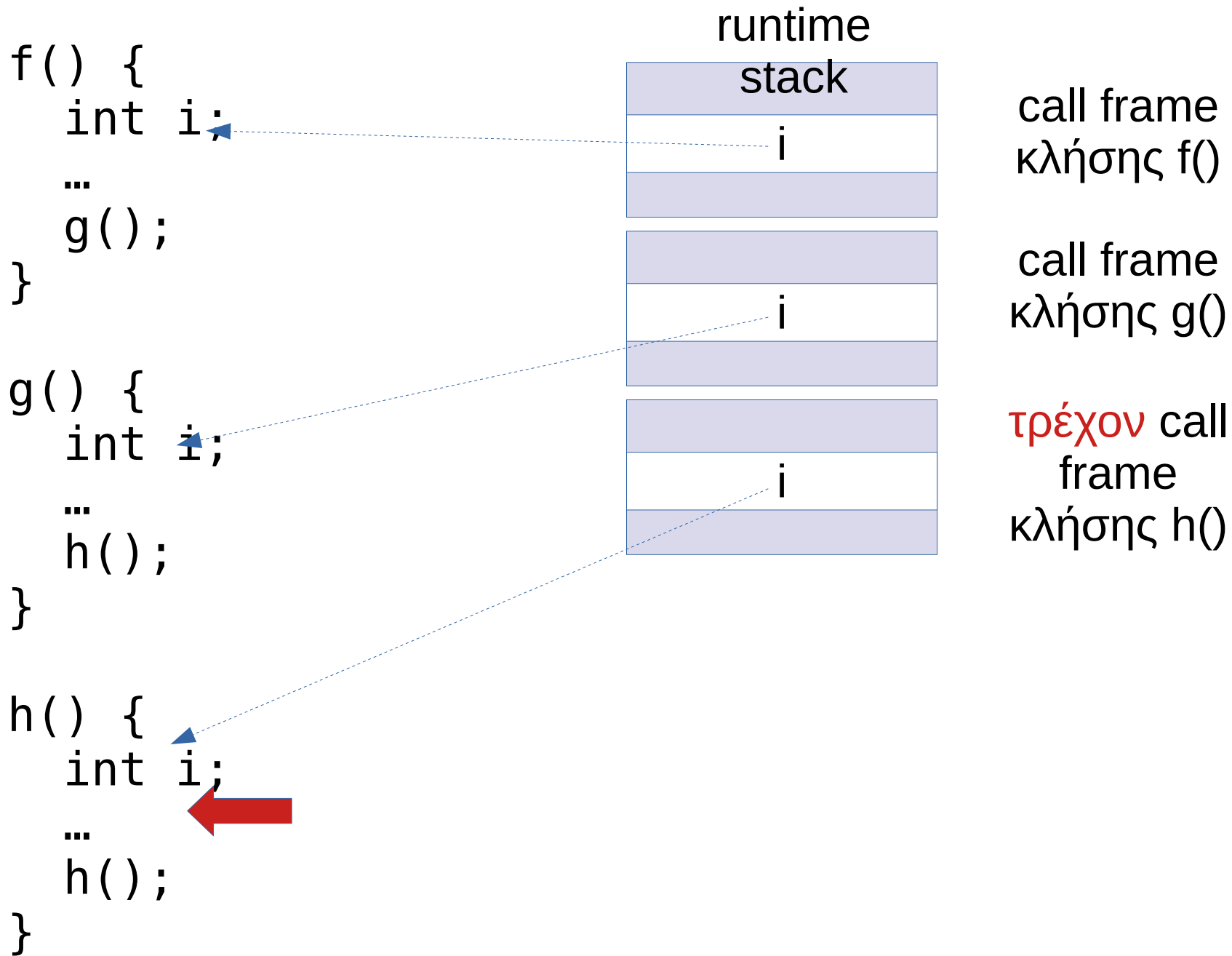
runtime



call frame
κλήσης f()

τρέχον call
frame
κλήσης g()

Παράδειγμα



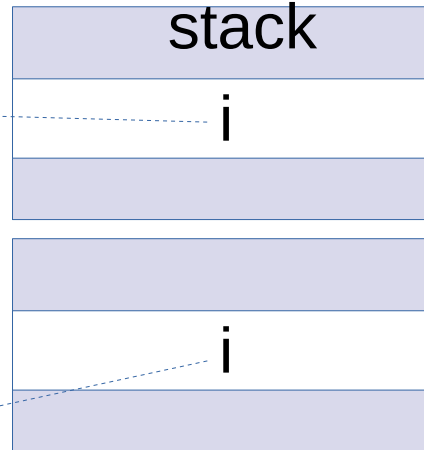
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

runtime

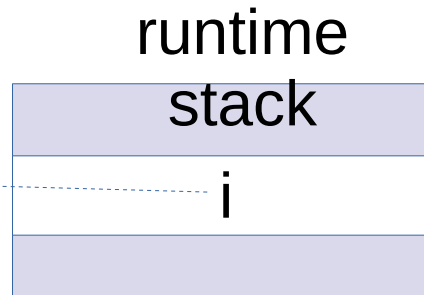


call frame
κλήσης f()

τρέχον call
frame
κλήσης g()

Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```



τρέχον call
frame
κλήσης f()

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

Κλάση RuntimeStack

- Θα τη βρείτε στις βιβλιοθήκες του εργαστηρίου
- Παρέχει ένα απλό API για τη διαχείριση ξεχωριστών χώρων συμβόλων (τοπικών μεταβλητών και των τιμών τους) ανά κλήση συνάρτησης

```
from compilerlabs import RuntimeStack
```

RuntimeStack API

- Δημιουργία νέου αντικειμένου RuntimeStack
`rs = RuntimeStack()`
 - Το νέο runtime stack θα αποτελείται από ένα μοναδικό call frame (αντιστοιχεί στο κυρίως πρόγραμμα)

RuntimeStack API (2)

- Προσθήκη νέου call frame

`rs.push_frame(zip(param_list, arg_values))`

- Το νέο call frame τοποθετείται στην κορυφή του runtime stack (γίνεται το τρέχον frame)
- Το προαιρετικό όρισμα του `push_frame()` αρχικοποιεί το νέο frame με τα ονόματα μεταβλητών του `param_list` και τις αντίστοιχες τιμές τους (`arg_values`)
 - Είναι λίστα από ζευγάρια (όνομα μεταβλητής, τιμή μεταβλητής)
 - Χρησιμοποιείται για να αρχικοποιήσουμε τις παραμέτρους των συναρτήσεων

RuntimeStack API (3)

- Διαγραφή τρέχοντος call frame

`rs.pop_frame()`

- Επιστροφή τιμής μεταβλητής στο τρέχον call frame

`value = rs.dereference(varname)`

– Εάν δεν υπάρχει το `varname` επιστρέφεται `None`

- Ανάθεση τιμής μεταβλητής στο τρέχον call frame

`rs.assign(varname, value)`

RuntimeStack API (4)

- Ανάθεση return value στο τρέχον call frame
`rs.return_value = value`
- Ανάκτηση return value από το τρέχον call frame
`rv = rs.return_value`
 - Εάν δεν έχει τεθεί ως τώρα, επιστρέφεται None
 - Προσοχή: η ανάκτηση του return_value πρέπει να γίνει **πρίν τη διαγραφή του τρέχοντος call frame**
 - δηλ. **πριν την κλήση** του pop_frame()

Διαδικασία κλήσης συνάρτησης

- **Έλεγχος**: υπάρχει το όνομα της συνάρτησης στον πίνακα συναρτήσεων;
- **Έλεγχος**: ο αριθμός των παραμέτρων στη δήλωση της συνάρτησης είναι ίσος με τον αριθμό των ορισμάτων κλήσης;
- **Υπολογισμός** κάθε ορίσματος κλήσης (subnodes στο FCALL)
- **Προσθήκη** (push) ενός νέου call frame, αρχικοποιημένου με τα ονόματα των παραμέτρων και τις αντίστοιχες τιμές των ορισμάτων κλήσης
- **Εκτέλεση** του AST της καλούμενης συνάρτησης
- **Αποθήκευση** της επιστρεφόμενης τιμής (return value)
- **Διαγραφή** (pop) του call frame
- **Επιστροφή** του return value

ControlFlow enumeration

- Χρειαζόμαστε έναν μηχανισμό για να διακόπτουμε την εκτέλεση μετά από **return**
 - Σε οποιοδήποτε βάθος (nesting level)
- Μέχρι τώρα η εκτέλεση των εντολών ενός block statement (συνάρτηση `execute_statements()`) δεν επέστρεφε κάποια τιμή
- Τώρα θα χρησιμοποιήσουμε την επιστρεφόμενη τιμή για να ελέγξουμε τη ροή εκτέλεσης μετά το `return`

```
from enum import Enum
```

```
ControlFlow = Enum('ControlFlow', 'NORMAL RETURN')
```

- Το προηγούμενο δημιουργεί τις τιμές enumeration:
 - `ControlFlow.NORMAL` → η εκτέλεση συνεχίζεται κανονικά
 - `ControlFlow.RETURN` → η εκτέλεση διακόπτεται, επιστροφή προς τα πίσω

Έλεγχος ροής ανά τύπο εντολής

ASSIGN PRINT	<i>(χωρίς διακοπή στη ροή εκτέλεσης)</i>
IF	<pre>if self.evaluate_expression(astnode.subnodes[0])!=0.0: cf = self.execute_statements(astnode.subnodes[1].subnodes) if cf==ControlFlow.RETURN: return cf</pre>
IFELSE	<pre>if self.evaluate_expression(astnode.subnodes[0])!=0.0: cf = self.execute_statements(astnode.subnodes[1].subnodes) else: cf = self.execute_statements(astnode.subnodes[2].subnodes) if cf==ControlFlow.RETURN: return cf</pre>
WHILE	<pre>while self.evaluate_expression(astnode.subnodes[0])!=0.0: cf = self.execute_statements(astnode.subnodes[1].subnodes) if cf==ControlFlow.RETURN: return cf</pre>
RETURN	<pre>return ControlFlow.RETURN</pre>